

Router Agent — Pseudocode & Token Budgeting

Purpose: Detailed pseudocode and operational design for the Router Agent (orchestrator) that composes context packages for the global LLM instance. Includes fragment ranking, token budgeting, staleness handling, continuation chaining, caching, and retries.

Goals & Responsibilities

- Given an active node and user query, produce a bounded context package for the LLM that contains the most relevant fragments (across nodes) plus compact metadata.
 - Respect a configurable token budget for the LLM call.
 - Prefer `fresh` nodes but include `stale` nodes' hallucination logs & prompt user revalidation when necessary.
 - Create clarification/continuation nodes when the Router detects missing data or token overflow.
 - Cache hot fragments and context packages for low-latency responses.
-

Configuration Parameters

```
MAX_PROMPT_TOKENS = 8192           # max tokens allowed by LLM for prompt
(example)
RESERVED_RESPONSE_TOKENS = 512      # tokens reserved for LLM output
DEFAULT_ROUTER_BUDGET = MAX_PROMPT_TOKENS - RESERVED_RESPONSE_TOKENS
FRAGMENT_MAX_TOKENS = 512          # max tokens we allow per fragment
MIN_FRAGMENT_SCORE = 0.1            # filter out low-similarity fragments
TOP_K_PER_NODE = 8                  # initial candidate fragments per node
GLOBAL_TOP_M = 40                   # top fragments accepted across all nodes
CACHE_TTL_SECONDS = 300             # cache lifetime for composed contexts
```

High-Level Steps

1. Parse explicit node references from query (e.g., `#Node_DB`).
2. Query Graph DB for candidate nodes: active node, dependencies, related nodes, recently-updated nodes.
3. For each candidate node, call `VectorService.search(node_id, query, top_k)` to get fragments with scores.
4. Score fragments by combined metric:
$$\text{final_score} = \alpha * \text{sim_score} + \beta * \text{recency_score} + \gamma * \text{node_priority}.$$
5. Filter out fragments below `MIN_FRAGMENT_SCORE`.

6. Rank fragments globally and pick top-N while respecting token budget (using estimated token counts per fragment).
 7. If token budget would be exceeded, summarize low-priority fragments (call LLM summarizer) or create continuation node.
 8. Build metadata header (node names, statuses, commit hashes, hallucination markers) and assemble context package.
 9. Cache context package and return to caller.
-

Helper utilities (assumed available)

- `token_count(text) -> int` — estimates token count for given text using tokenizer matching LLM.
 - `embed(text) -> vector` — compute embedding for a text fragment.
 - `vector_search(vector, index_id, top_k) -> List[(fragment_id, text, score, commit_id)]` — returns candidate fragments.
 - `summarize_texts(texts, token_budget) -> summary_text` — LLM call to compress many fragments into a short summary.
 - `create_continuation_node(parent_node_id, reason, initial_fragments)` — creates continuation node with references.
 - `cache.get(key) / cache.set(key, value, ttl)`
-

Detailed Pseudocode

```
def route_query(active_node_id: str, user_query: str, requested_budget: int =
DEFAULT_ROUTER_BUDGET) -> ContextPackage:
    # 1. Normalize budget
    budget = min(requested_budget, DEFAULT_ROUTER_BUDGET)

    # 2. Check cache (cache key based on node+query+budget)
    cache_key = f"ctx:{active_node_id}:{hash_query(user_query)}:{budget}"
    cached = cache.get(cache_key)
    if cached:
        return cached

    # 3. Parse explicit references
    explicit_node_ids = parse_explicit_node_refs(user_query) # e.g., #Node_DB

    # 4. Candidate node collection (Graph traversal)
    candidates = set()
    candidates.add(active_node_id)
    candidates.update(explicit_node_ids)

    # Add dependencies (depth-limited) and related nodes
```

```

deps = GraphDB.fetch_dependencies(active_node_id, depth=2)
related = GraphDB.fetch_related(active_node_id)
recent = GraphDB.fetch_recent_nodes(limit=10)

candidates.update(deps)
candidates.update(related)
candidates.update(recent)

# 5. For each candidate node, fetch top-k fragments
fragment_candidates = [] # list of dicts: {node_id, frag_id, text, score,
commit_id, created_at}

for node_id in candidates:
    # skip archived nodes
    node = GraphDB.get_node(node_id)
    if node.status == 'archived':
        continue

    top_frags = VectorService.search(node_id, user_query,
top_k=TOP_K_PER_NODE)
    for frag in top_frags:
        # compute recency boost and node_priority
        recency_score = recency_boost(frag['created_at'])
        node_priority = node_priority_weight(node)
        final_score = 0.6 * frag['score'] + 0.3 * recency_score + 0.1 *
node_priority
        if final_score >= MIN_FRAGMENT_SCORE:
            fragment_candidates.append({
                'node_id': node_id,
                'frag_id': frag['fragment_id'],
                'text': frag['text'],
                'score': final_score,
                'commit_id': frag['commit_id'],
                'created_at': frag['created_at']
            })

# 6. Sort globally by final_score desc
fragment_candidates.sort(key=lambda f: f['score'], reverse=True)

# 7. Greedy selection under token budget
selected_fragments = []
tokens_used = token_count(build_metadata_header(active_node_id, candidates))
tokens_limit = budget

for frag in fragment_candidates[:GLOBAL_TOP_M]:
    frag_tokens = min(FRAGMENT_MAX_TOKENS, token_count(frag['text']))
    if tokens_used + frag_tokens > tokens_limit:
        # not enough room for this fragment

```

```

        # try to compress remaining low-priority fragments into a summary
        remaining =
fragment_candidates[fragment_candidates.index(frag):fragment_candidates.index(frag)
+10]
        if remaining:
            summary = summarize_texts([r['text'] for r in remaining],
token_budget=(tokens_limit - tokens_used))
            if summary:
                selected_fragments.append({'node_id': active_node_id,
'frag_id': 'summary', 'text': summary, 'score': 0.5, 'commit_id': None})
                tokens_used += token_count(summary)
            break

        selected_fragments.append(frag)
        tokens_used += frag_tokens

    # 8. If no fragments could be included due to tiny budget, create
continuation node
    if len(selected_fragments) == 0:
        cont_node_id = create_continuation_node(active_node_id,
reason='token_budget_exceeded', initial_fragments=fragment_candidates[:10])
        # return a minimal package with a pointer to continuation
        package = ContextPackage(
            active_node=active_node_id,
            selected_fragments=[{'node_id': active_node_id, 'frag_id':
'CONTINUATION', 'text': f'CONTINUATION_NODE:{cont_node_id}'},],
            metadata={'nodes': build_node_metadata(candidates)})
        )
        cache.set(cache_key, package, ttl=CACHE_TTL_SECONDS)
        return package

    # 9. Build metadata header
    metadata = build_node_metadata(candidates)

    # 10. Assemble context package
    package = ContextPackage(
        active_node=active_node_id,
        selected_fragments=selected_fragments,
        metadata=metadata,
        user_query=user_query
    )

    # 11. Cache and return
    cache.set(cache_key, package, ttl=CACHE_TTL_SECONDS)
    return package

```

Staleness & Hallucination Handling

- If a candidate node is `stale`, the Router includes its `hallucination_log` items (short summaries) in the metadata and sets a `requires_validation` flag in the package. The frontend will surface a validation prompt to user before merging any stale content into main aggregation.
 - If a node has a recent high frequency of hallucinations, lower its `node_priority_weight` to reduce its fragments' inclusion probability.
-

Continuation Node Strategy

- When a node's context exceeds token quotas, Router should create a `Continuation` node (type=continuation) with a pointer chain back to the parent and copy top fragments into it.
 - The Router should prefer retrieving fragments from the latest continuation nodes when the parent context is truncated.
-

Caching Strategy

- Cache context packages keyed by `node_id+query_hash+budget`.
 - Cache most-recent vector search results per node-query pair for a short TTL.
 - Invalidate caches when a parent node is updated and snapshots are created (emit event via AgentBus).
-

Failure modes and retries

- VectorService or GraphDB transient errors: retry with exponential backoff up to 3 attempts.
 - LLM summarization failures: fall back to including raw fragments until retry is possible; mark package `partial=true`.
 - If cache is unavailable, continue with live retrieval.
-

Observability & Metrics

- Track: `time_to_compose` (ms), `fragments_fetched`, `fragments_selected`, `tokens_used`, `cache_hit_rate`, `staleness_included` (bool), `continuation_created` (bool).
 - Log decisions for audit (why a fragment was chosen/dropped).
-

Example token budgeting scenario

- `MAX_PROMPT_TOKENS=8192`, `RESERVED_RESPONSE_TOKENS=512` → `BUDGET=7680`
- Metadata header estimated 200 tokens

- Selected fragments limited to 7400 tokens; with FRAGMENT_MAX_TOKENS=512 → up to ~14 fragments if uncompressed
 - Use summarization to pack many low-score fragments into 1–2 summaries when needed.
-

End of Router Agent pseudocode document.